

1. Decision Control Statements: Explain the concept of decision control statements in Python. Discuss the syntax and usage of the following:

- **if, else, and elif statements**
- **How these statements are used to execute different blocks of code based on certain conditions. Provide an example where if, else, and elif are used to categorize an integer as positive, negative, or zero.**

Decision Control Statements in Python

Decision control statements allow the execution of specific blocks of code based on certain conditions. These include **if**, **else**, and **elif** statements.

Syntax:

1. **if statement:** Evaluates a condition and executes the code block if the condition is true.

```
python
Copy code
if condition:
    # Code block
```

2. **else statement:** Executes a code block when the condition in the `if` statement is false.

```
python
Copy code
if condition:
    # Code block
else:
    # Code block
```

3. **elif statement:** Used for multiple conditions. It stands for "else if".

```
python
Copy code
if condition1:
    # Code block
elif condition2:
    # Code block
else:
    # Code block
```

Usage:

- **if, else, and elif** are used to control the flow of the program by checking conditions and executing corresponding blocks of code.

Example: Categorizing an integer as positive, negative, or zero

```
python
Copy code
number = int(input("Enter an integer: "))

if number > 0:
    print("The number is positive.")
```

```
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Explanation:

1. **if** checks if `number > 0` and executes the block if true.
2. **elif** checks another condition (`number < 0`) if the first is false.
3. **else** executes when none of the previous conditions are true.

This ensures the integer is categorized correctly.

2. Looping and Nested Loops: Discuss the difference between while loops and for loops in Python.

- Describe how both loops work and when to use each.
- Explain the concept of nested loops and provide an example of a program that uses a nested loop to print a multiplication table

Looping and Nested Loops in Python

Loops are used to execute a block of code repeatedly.

Difference Between `while` and `for` Loops:

1. **while loop:** Repeats as long as a condition is true.
 - Use when the number of iterations is unknown.
 - Syntax:

```
python
Copy code
while condition:
    # Code block
```

2. **for loop:** Iterates over a sequence (list, range, etc.).
 - Use when the number of iterations is predetermined.
 - Syntax:

```
python
Copy code
for item in sequence:
    # Code block
```

Nested Loops:

A nested loop is a loop inside another loop. The inner loop runs completely for each iteration of the outer loop.

Example: Multiplication Table Using Nested Loops

```
python
Copy code
rows = 5
```

```
cols = 5

for i in range(1, rows + 1):      # Outer loop for rows
    for j in range(1, cols + 1):  # Inner loop for columns
        print(i * j, end="\t")    # Print product
    print()                       # Move to the next row
```

Explanation:

1. **Outer Loop:** Iterates over rows (1 to `rows`).
2. **Inner Loop:** Iterates over columns (1 to `cols`) for each row.
3. The multiplication of `i` (row) and `j` (column) prints the table.

Output for `rows = 5` and `cols = 5`:

Copy code

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
5 10 15 20 25
```

3. List, Dictionary, and Set Comprehensions: Define comprehensions in Python.

- Explain list comprehensions, dictionary comprehensions, and set comprehensions.
- Discuss their syntax, how they can be used to create new collections in a concise way, and provide examples to generate:
 - A list of squares of even numbers between 1 and 10.
 - A dictionary of square roots of integers from 1 to 5
 - A set containing only unique letters from a string.

Comprehensions in Python

Comprehensions are a concise way to create lists, dictionaries, or sets from iterables. They combine loops and conditional statements in a single line.

List Comprehension

- **Syntax:**

python
Copy code
[expression for item in iterable if condition]

- **Example:** Squares of even numbers between 1 and 10:

python
Copy code
squares = [x**2 for x in range(1, 11) if x % 2 == 0]
print(squares) # Output: [4, 16, 36, 64, 100]

Dictionary Comprehension

- **Syntax:**

```
python
Copy code
{key_expression: value_expression for item in iterable if condition}
```

- **Example:** Square roots of integers from 1 to 5:

```
python
Copy code
import math
sqrt_dict = {x: math.sqrt(x) for x in range(1, 6)}
print(sqrt_dict) # Output: {1: 1.0, 2: 1.414, 3: 1.732, 4: 2.0, 5: 2.236}
```

Set Comprehension

- **Syntax:**

```
python
Copy code
{expression for item in iterable if condition}
```

- **Example:** Unique letters from a string:

```
python
Copy code
unique_letters = {char for char in "comprehensions" if char.isalpha()}
print(unique_letters) # Output: {'c', 'o', 'm', 'p', 'r', 'e', 'h', 'n', 's', 'i'}
```

4. Arrays: 1D and 2D Arrays in Python: Explain the concept of arrays in Python, focusing on:

- How arrays differ from lists in Python
- How to create and manipulate 1D arrays using the array module
- How to work with 2D arrays using the numpy module
- Provide examples to demonstrate how to create, initialize, and access elements in both 1D and 2D arrays.

Arrays in Python

An **array** is a collection of items stored at contiguous memory locations. Unlike lists, arrays are more efficient for numerical operations and require elements to be of the same data type.

Difference Between Arrays and Lists

1. **Arrays:** Require all elements to be of the same type and support numerical operations.
 2. **Lists:** Can hold elements of mixed data types but are slower for numerical computations.
-

1D Arrays Using the `array` Module

- **Creation:**

```
python
Copy code
from array import array
arr = array('i', [1, 2, 3, 4]) # 'i' indicates integer type
print(arr) # Output: array('i', [1, 2, 3, 4])
```

- **Manipulation:**

```
python
Copy code
arr.append(5) # Add element
arr.pop(2) # Remove element at index 2
print(arr[1]) # Access element at index 1
```

2D Arrays Using the `numpy` Module

- **Creation:**

```
python
Copy code
import numpy as np
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr_2d) # Output: [[1 2 3]
                #         [4 5 6]]
```

- **Accessing Elements:**

```
python
Copy code
print(arr_2d[0, 1]) # Access element at row 0, column 1 (Output: 2)
print(arr_2d[:, 1]) # Access all rows, column 1 (Output: [2 5])
```

- **Initialization:**

```
python
Copy code
zeros = np.zeros((2, 3)) # 2x3 array of zeros
ones = np.ones((2, 3)) # 2x3 array of ones
```

Example: Creating and Accessing Arrays

1D Array:

```
python
Copy code
from array import array
arr = array('f', [1.1, 2.2, 3.3]) # Float array
print(arr[0]) # Output: 1.1
```

2D Array:

```
python
Copy code
import numpy as np
arr_2d = np.array([[10, 20], [30, 40]])
print(arr_2d[1, 0]) # Output: 30
```

5. Functions and Recursion: Describe the importance of functions in Python.

- Explain how functions help in code modularization and reuse.
- Discuss function parameters, return values, and scope of variables within functions.
- Define recursion and explain how recursive functions work, providing an example of a function to compute the factorial of a number.

Functions in Python

Functions are reusable blocks of code that perform specific tasks. They help in **code modularization** and **reuse**, making programs easier to read, debug, and maintain.

Benefits of Functions

1. **Modularization:** Breaks a program into smaller, manageable pieces.
2. **Reusability:** Functions can be called multiple times, reducing code duplication.

Function Components

1. **Parameters:** Variables passed into a function.

```
python
Copy code
def greet(name): # 'name' is a parameter
    print(f"Hello, {name}!")
```

2. **Return Values:** Functions can return results to the caller.

```
python
Copy code
def add(a, b):
    return a + b
result = add(3, 5) # result = 8
```

3. Scope of Variables:

- **Local Scope:** Variables defined inside a function.
- **Global Scope:** Variables accessible throughout the program.

```
python
Copy code
x = 10 # Global
def func():
    y = 5 # Local
```

Recursion

Recursion is a function calling itself to solve smaller instances of the same problem. A **base case** is essential to stop recursion.

Example: Factorial Calculation

```
python
Copy code
def factorial(n):
    if n == 0: # Base case
        return 1
    else:
        return n * factorial(n - 1) # Recursive call

print(factorial(5)) # Output: 120
```

How Recursive Functions Work

1. The function keeps calling itself with smaller arguments.
 2. When the base case is reached, recursion stops and results are returned step-by-step.
-

6. Lambda Functions: Define lambda functions in Python.

- Explain how they differ from regular functions defined using the `def` keyword.
- Discuss their syntax, when to use them, and provide an example that demonstrates a lambda function to square a number.

Lambda Functions in Python

A **lambda function** is a small, anonymous function defined using the `lambda` keyword. It is typically used for short, simple operations.

Differences from Regular Functions

1. **Syntax:** Lambda functions are written in a single line, unlike regular functions.
2. **Name:** Lambdas are anonymous (do not require a name) unless assigned to a variable.

3. **Use Case:** Lambdas are used for quick, short-lived functions, often passed as arguments to higher-order functions.
-

Syntax

```
python
Copy code
lambda arguments: expression
```

- **Arguments:** Input parameters for the function.
 - **Expression:** A single expression whose result is returned.
-

When to Use Lambda Functions

- When a short, throwaway function is needed.
 - Commonly used with functions like `map()`, `filter()`, and `sorted()`.
-

Example: Lambda to Square a Number

```
python
Copy code
square = lambda x: x**2
print(square(5)) # Output: 25
```

Use in Higher-Order Functions

Example with `map()` to square numbers in a list:

```
python
Copy code
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, nums))
print(squares) # Output: [1, 4, 9, 16]
```

7. Searching Techniques: Linear and Binary Search: Explain the concepts of Linear Search and Binary Search.

- **Discuss the algorithm and time complexity of both techniques.**
 - Describe the differences between the two, including when each search method should be used.
- **Provide theoretical examples of how each search algorithm works.**

Searching Techniques: Linear and Binary Search

1. Linear Search

- **Concept:** Sequentially checks each element in the list until the target value is found or the end of the list is reached.
- **Algorithm:**
 1. Start from the first element.
 2. Compare each element with the target.
 3. Return the index if a match is found; otherwise, continue to the next element.
 4. If no match is found, return failure.
- **Time Complexity:**
 - Best case: $O(1)$ (target is the first element).
 - Worst case: $O(n)$ (target is not present, or is the last element).
- **When to Use:** Suitable for unsorted or small datasets.

Example: Search for 5 in [3, 8, 5, 2]:

- Check 3 → 8 → 5 → Found at index 2.
-

2. Binary Search

- **Concept:** Works on sorted lists. Divides the list into halves to locate the target.
- **Algorithm:**
 1. Check the middle element.
 2. If the target equals the middle, return the index.
 3. If the target is smaller, search in the left half.
 4. If the target is larger, search in the right half.
 5. Repeat until the target is found or the sub-list is empty.
- **Time Complexity:**
 - Best case: $O(1)$ (middle element is the target).
 - Worst case: $O(\log n)$ (repeatedly halving the list).
- **When to Use:** Suitable for large, sorted datasets.

Example: Search for 8 in [2, 4, 6, 8, 10]:

- Middle = 6 → Target > 6 → Search [8, 10].
 - Middle = 8 → Found at index 3.
-

Key Differences

Feature	Linear Search	Binary Search
Dataset	Works on unsorted/sorted	Requires sorted data
Time Complexity	$O(n)$	$O(\log n)$
Implementation	Simple	More complex
Use Case	Small/unsorted data	Large/sorted data

Summary

- Use **Linear Search** for small or unsorted datasets.
- Use **Binary Search** for large, sorted datasets for faster lookups.

8. Sorting Techniques: Selection Sort and Bubble Sort: Define and explain the Selection Sort and Bubble Sort algorithms.

- Describe the steps involved in each sorting algorithm.
- Compare the time and space complexities of both algorithms.
- Provide a theoretical example of how each algorithm sorts a given list of integers in ascending order.

Sorting Techniques: Selection Sort and Bubble Sort

1. Selection Sort

- **Definition:** A sorting algorithm that repeatedly selects the smallest element from the unsorted portion of the list and swaps it with the first unsorted element.

Steps:

1. Start with the first element.
2. Find the smallest element in the unsorted portion.
3. Swap it with the current element.
4. Move to the next position and repeat until the list is sorted.

Time Complexity:

- Best, Worst, and Average Case: $O(n^2)$
- **Space Complexity:** $O(1)$ (in-place sorting).

Example: Sort [3, 1, 4, 2]:

1. Find smallest (1) → Swap with 3 → [1, 3, 4, 2].
 2. Find smallest (2) → Swap with 3 → [1, 2, 4, 3].
 3. Find smallest (3) → Swap with 4 → [1, 2, 3, 4].
-

2. Bubble Sort

- **Definition:** A sorting algorithm that repeatedly swaps adjacent elements if they are in the wrong order.

Steps:

1. Compare adjacent elements.
2. Swap if the first is larger than the second.
3. Continue this process for all elements.
4. Repeat for remaining elements until the list is sorted.

Time Complexity:

- Best Case: $O(n)$ (already sorted).
- Worst and Average Case: $O(n^2)$.
- **Space Complexity:** $O(1)$ (in-place sorting).

Example: Sort [3, 1, 4, 2]:

1. Compare 3 & 1 → Swap → [1, 3, 4, 2].
2. Compare 3 & 4 → No swap → [1, 3, 4, 2].
3. Compare 4 & 2 → Swap → [1, 3, 2, 4].
4. Repeat until [1, 2, 3, 4].

Comparison

Feature	Selection Sort	Bubble Sort
Process	Select smallest and swap	Swap adjacent elements
Best Case	$O(n)$	$O(n)$
Worst Case	$O(n^2)$	$O(n^2)$
Stability	Not stable	Stable
Space	$O(1)$	$O(1)$

Summary

- **Selection Sort** is simpler but inefficient for large lists.
- **Bubble Sort** is useful when the list is nearly sorted.

1. Decision Control Statements: Write a Python program to determine whether a given number is prime or composite using if, elif, and else statements. The program should prompt the user to input a number and display whether the number is prime, composite, or neither (e.g., if the number is 1).

Python Program: Check Prime, Composite, or Neither

```
python
Copy code
# Prompt user for input
num = int(input("Enter a number: "))

# Check conditions using if, elif, and else
```

```

if num <= 1:
    print("The number is neither prime nor composite.")
elif num == 2: # 2 is the smallest prime number
    print("The number is prime.")
else:
    # Check divisibility
    is_prime = True
    for i in range(2, int(num**0.5) + 1): # Check up to the square root of
the number
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print("The number is prime.")
    else:
        print("The number is composite.")

```

Explanation:

1. Input Handling:

- If the number is ≤ 1 , it's neither prime nor composite.
- If the number is 2, it's prime.

2. Prime Check:

- Iterate from 2 to $\sqrt{\text{num}}$.
- If the number is divisible by any, it's composite; otherwise, it's prime.

3. Output: Displays whether the number is prime, composite, or neither.

Example:

- Input: 5 → Output: "The number is prime."
- Input: 9 → Output: "The number is composite."
- Input: 1 → Output: "The number is neither prime nor composite."

2. Looping and Nested Loops: Write a Python program that prints the Floyd's Triangle using nested for loops. The program should take an integer n as input and print the first n rows of the triangle. The triangle should look like this for n = 5: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Python Program: Floyd's Triangle

```

python
Copy code
# Input: Number of rows
n = int(input("Enter the number of rows: "))

# Initialize starting number
num = 1

# Nested loop to print Floyd's Triangle
for i in range(1, n + 1): # Outer loop for rows
    for j in range(1, i + 1): # Inner loop for columns in each row
        print(num, end=" ")
        num += 1

```

```
print() # Move to the next row
```

Explanation:

1. **Outer Loop:** Iterates over the rows from 1 to n.
 2. **Inner Loop:** Prints the numbers for each row, with the count increasing in every iteration.
 3. **Number Update:** The variable `num` is incremented after printing each number.
 4. **Row Formatting:** `print()` at the end of the outer loop moves to the next row.
-

Example Output:

For n = 5:

Copy code

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

3. List, Dictionary, and Set Comprehensions: Write a Python program using list comprehension, dictionary comprehension, and set comprehension to:

- Create a list of all the even numbers between 1 and 50.
- Create a dictionary where keys are numbers from 1 to 5 and values are their squares.
- Create a set of unique characters from the string "programming".

Python Program: List, Dictionary, and Set Comprehensions

python

Copy code

```
# List comprehension: Even numbers between 1 and 50
even_numbers = [x for x in range(1, 51) if x % 2 == 0]
print("Even numbers:", even_numbers)

# Dictionary comprehension: Numbers (1 to 5) as keys and their squares as values
squares_dict = {x: x**2 for x in range(1, 6)}
print("Squares dictionary:", squares_dict)

# Set comprehension: Unique characters from the string "programming"
unique_chars = {char for char in "programming"}
print("Unique characters:", unique_chars)
```

Explanation:

1. **List Comprehension:**
 - Iterates through numbers from 1 to 50.
 - Includes only numbers divisible by 2 (even numbers).

2. Dictionary Comprehension:

- Keys are numbers from 1 to 5.
- Values are the squares of the keys.

3. Set Comprehension:

- Extracts unique characters from the string "programming".
 - The set automatically removes duplicates.
-

Output Example:

```
sql
Copy code
Even numbers: [2, 4, 6, 8, 10, ..., 48, 50]
Squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
Unique characters: {'r', 'o', 'g', 'm', 'i', 'n', 'p', 'a'}
```

4. Arrays: 1D and 2D Arrays in Python: Write a Python program that:

- Creates a 1D array using the array module and initializes it with integers.
- Creates a 2D array using the numpy module and fills it with random numbers between 1 and 100.
- Accesses the element at position (1, 2) in the 2D array and prints it.

Python Program: 1D and 2D Arrays

```
python
Copy code
# 1D Array using the array module
from array import array
arr_1d = array('i', [10, 20, 30, 40, 50]) # Integer array
print("1D Array:", arr_1d)

# 2D Array using the numpy module
import numpy as np
arr_2d = np.random.randint(1, 101, size=(3, 4)) # 3x4 array with random
integers (1-100)
print("2D Array:\n", arr_2d)

# Access element at position (1, 2)
element = arr_2d[1, 2]
print("Element at position (1, 2):", element)
```

Explanation:

1. 1D Array:

- Created using the array module.
- Initialized with integers [10, 20, 30, 40, 50].

2. 2D Array:

- Created using numpy's random.randint() function.
- Generates a 3x4 array with random numbers between 1 and 100.

3. Accessing an Element:

- Use arr_2d[row, column] to access elements.

- Position (1, 2) corresponds to the second row, third column (indexing starts from 0).

Example Output:

```
lua
Copy code
1D Array: array('i', [10, 20, 30, 40, 50])
2D Array:
[[25 89 74 50]
 [12 36 58 99]
 [65 48 73 20]]
Element at position (1, 2): 58
```

5. Functions and Recursion: Write a Python program that defines a recursive function to compute the factorial of a given number n. Additionally, implement a non-recursive version of the factorial calculation in a separate function

Python Program: Factorial (Recursive and Non-Recursive)

```
python
Copy code
# Recursive function to compute factorial
def factorial_recursive(n):
    if n == 0 or n == 1: # Base case: 0! and 1! are both 1
        return 1
    else:
        return n * factorial_recursive(n - 1) # Recursive call

# Non-recursive (iterative) function to compute factorial
def factorial_non_recursive(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Input
num = int(input("Enter a number: "))

# Output for recursive factorial
print(f"Factorial of {num} (recursive):", factorial_recursive(num))

# Output for non-recursive factorial
print(f"Factorial of {num} (non-recursive):", factorial_non_recursive(num))
```

Explanation:

- 1. Recursive Function (factorial_recursive):**
 - The function calls itself to calculate the factorial.
 - The base case handles 0! and 1!, returning 1.
 - For other numbers, it multiplies n by the factorial of n-1.
- 2. Non-Recursive Function (factorial_non_recursive):**

- Uses a loop to calculate the factorial iteratively.
 - Initializes `result` to 1 and multiplies it by each number from 1 to `n`.
3. **Input and Output:**
- The program prompts the user to enter a number.
 - It then calculates and prints the factorial using both recursive and non-recursive methods.
-

Example Output:

```
mathematica
Copy code
Enter a number: 5
Factorial of 5 (recursive): 120
Factorial of 5 (non-recursive): 120
```

6. Lambda Functions: Write a Python program that uses a lambda function to:

- Take a list of integers as input from the user.
- Use the lambda function to filter out the even numbers from the list.
- Print the list of even numbers. Give answer for this question in short but everything need to be covered

Python Program: Filter Even Numbers Using a Lambda Function

```
python
Copy code
# Input: List of integers from the user
nums = list(map(int, input("Enter a list of integers separated by spaces: ").split()))

# Lambda function to filter even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, nums))

# Output: List of even numbers
print("Even numbers:", even_numbers)
```

Explanation:

1. **Input Handling:**
 - `input()` takes a space-separated list of integers.
 - `map(int, input().split())` converts the input into a list of integers.
2. **Lambda Function:**
 - `lambda x: x % 2 == 0` is a small, anonymous function that checks if a number is even.
3. **Filter Function:**
 - `filter()` applies the lambda to each element of the list and keeps only the even numbers.
 - The result is converted to a list using `list()`.
4. **Output:**
 - Prints the filtered list of even numbers.

Example Output:

```
less
Copy code
Enter a list of integers separated by spaces: 1 2 3 4 5 6
Even numbers: [2, 4, 6]
```

7. Searching Techniques: Linear and Binary Search: Write two separate functions:

- A linear search function to find an element in an unsorted list.
- A binary search function to find an element in a sorted list. For both functions, the program should display the index of the element if found, or a message saying the element was not found

Python Program: Linear and Binary Search

```
python
Copy code
# Linear Search Function
def linear_search(lst, target):
    for i in range(len(lst)):
        if lst[i] == target:
            return f"Element found at index {i}"
    return "Element not found"

# Binary Search Function
def binary_search(lst, target):
    low, high = 0, len(lst) - 1
    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return f"Element found at index {mid}"
        elif lst[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return "Element not found"

# Input for Linear Search
unsorted_list = [5, 3, 8, 6, 2]
target1 = int(input("Enter the element to search in unsorted list: "))
print(linear_search(unsorted_list, target1))

# Input for Binary Search
sorted_list = [1, 3, 5, 7, 9]
target2 = int(input("Enter the element to search in sorted list: "))
print(binary_search(sorted_list, target2))
```

Explanation:

1. Linear Search:

- Iterates through the list.
- Checks each element against the target.
- Time Complexity: $O(n)$

- Works on **unsorted lists**.
 - 2. **Binary Search:**
 - Works on sorted lists.
 - Divides the list into halves to locate the target.
 - Time Complexity: $O(\log_{10} n)$ $O(\log n)$ $O(\log n)$.
 - 3. **Input/Output:**
 - For both functions, the user provides the list and the target value.
 - Outputs the index if the target is found or a message if it is not.
-

Example Output:

Linear Search:

```

mathematica
Copy code
Enter the element to search in unsorted list: 8
Element found at index 2

```

Binary Search:

```

mathematica
Copy code
Enter the element to search in sorted list: 5
Element found at index 2

```

8. Sorting Techniques: Selection Sort and Bubble Sort: Write a Python program that implements both Selection Sort and Bubble Sort algorithms to sort a given list of integers in ascending order. The program should:

- Take a list of integers as input from the user.
- Sort the list using both sorting techniques.
- Print the sorted list for both sorting algorithms.

Python Program: Selection Sort and Bubble Sort

```

python
Copy code
# Selection Sort Function
def selection_sort(lst):
    n = len(lst)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if lst[j] < lst[min_index]:
                min_index = j
        lst[i], lst[min_index] = lst[min_index], lst[i] # Swap
    return lst

# Bubble Sort Function
def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lst[j] > lst[j + 1]:

```

```

        lst[j], lst[j + 1] = lst[j + 1], lst[j] # Swap
    return lst

# Input: List of integers
input_list = list(map(int, input("Enter a list of integers separated by
spaces: ").split()))

# Apply Selection Sort
selection_sorted = selection_sort(input_list.copy())
print("Sorted list using Selection Sort:", selection_sorted)

# Apply Bubble Sort
bubble_sorted = bubble_sort(input_list.copy())
print("Sorted list using Bubble Sort:", bubble_sorted)

```

Explanation:

1. Selection Sort:

- Finds the smallest element and swaps it with the first unsorted element.
- Repeats for the rest of the list.
- Time Complexity: $O(n^2)$

2. Bubble Sort:

- Repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- Time Complexity: $O(n^2)$

3. Input/Output:

- The user provides a list of integers.
 - Both algorithms sort the list and display the sorted output.
-

Example Output:

```

mathematica
Copy code
Enter a list of integers separated by spaces: 5 2 9 1 6
Sorted list using Selection Sort: [1, 2, 5, 6, 9]
Sorted list using Bubble Sort: [1, 2, 5, 6, 9]

```